PilotFish File Writing Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **save-to-disk** step. Whatever bytes are currently flowing through the transaction get written to a folder and file name you configure, while the route keeps the same content available for downstream processors.

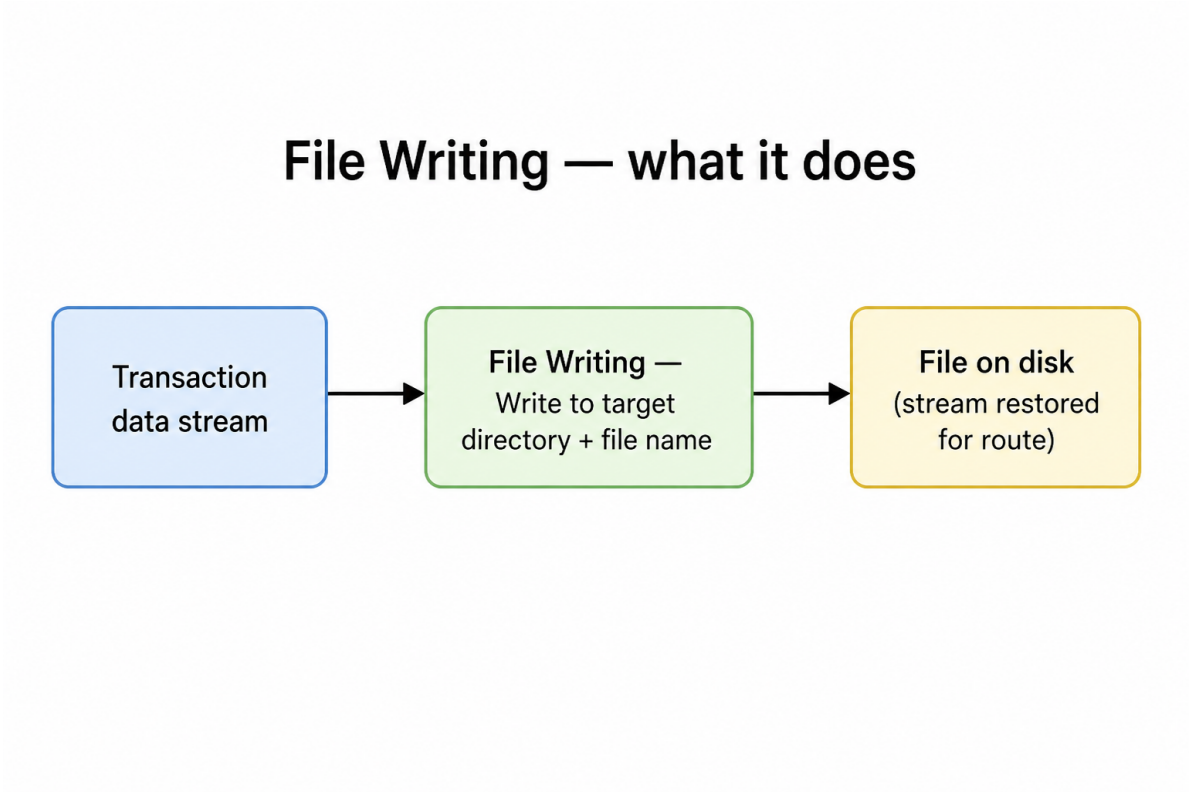

Figure 1: At a glance (plain English)

- **Writes** the full transaction stream to a target directory and file name.
- **Creates** missing directories automatically (`mkdirs`).
- **Restores** the stream from an internal cache so later stages still see the payload.
- **Evaluates** directory and file name through the configuration expression engine.

Purpose

This document is a **deep dive** into the **File Writing** processor as shipped in **`eip.war.hs.26R1.11`**: how it resolves paths, copies bytes to disk, and rewinds the transaction stream from the EIP default cache.

Provenance	Value
WAR	`eip.war.hs.26R1.11`
Module JAR	`xcs-modules-file-1.0.1.jar`
Decompiler	CFR 0.152
Implementation class	`com.pilotfish.eip.modules.file.FileWritePr`
Decompiled path	`war-pipeline/decompiled/eip-26R1.11/.../Fi`
Processor type string	`File Writing`
Description	`Copies the transaction contents to the specified file path`
Help ID	`console.processor.FileWriteProcessor`
Categories	`FILES`

Derived from WAR bytecode (CFR). Narrative follows executable logic in `processData`.

---

1. Conceptual model

File Writing is a **side-effect + stream preservation** processor:

```
Transaction data stream
        |
        v
 EIPCacheManager default Cache  <-- copy #1 (consume original stream)
        |
        v
 FileOutputStream(target dir + file name)  <-- copy #2 to disk
        |
        v
 data.setDataStream(cache.getInputStream())  <-- restored for route
```

The on-disk file and the in-route stream contain the same bytes after success.

---

2. High-level behavior (`processData`)

1. **Resolve target directory**
   - `manager.getFile("TargetDirectory", data, loader)` -> `File directory`
   - `directory.mkdirs()` — creates the full path if missing (no error if already exists).
2. **Resolve file name**
   - `new File(directory, manager.getStringValue("TargetFileName", data))`

- File name is expression-evaluated; can include path segments relative to the directory depending on platform (see quirks).

3. **Buffer transaction bytes**
   - `EIPCacheManager.getInstance().getDefaultCache(data.getDataStream().available())`
   - `DataUtil.copyAndClose(data.getDataStream(), cache.getOutputStream())`
4. **Write to disk**
   - `DataUtil.copyAndClose(cache.getInputStream(), new FileOutputStream(file))`
5. **Restore route stream**
   - `data.setDataStream(cache.getInputStream())`
6. **Return** the same `TransactionData`.
7. **Any exception** -> `EIPException("Error writing file", cause)`.

---

3. Configuration reference

3.1 Target Directory

Property	Value
UI label	Target Directory
Tag	`TargetDirectory`
Type	Path
Default	empty string

Nuances:

- Resolved via `ConfigurationManager.getFile` — supports interface-relative paths through `ResourceLoader`.
- `mkdirs()` runs before write; permissions and disk space failures surface as wrapped I/O exceptions.
- Required for a successful write in practice (empty path behavior depends on `getFile` resolution).

3.2 File Name

Property	Value
UI label	File Name
Tag	`TargetFileName`
Type	String
Default	empty string

Nuances:

- Evaluated per transaction — typical pattern: `%com.pilotfish.FileName%` or timestamp expressions.
- Combined with directory as `new File(directory, fileName)` — embedded `/` or `\` in the name creates subpaths under the target directory.

- Empty name yields a directory entry named `""` (usually undesirable); validate in route design.

3.3 Inherited AbstractProcessor options

Property	Value
UI label	Execute Processor
Tag	`ExecuteProcessor`
Type	Boolean
Default	`true`
Tab	Conditional Execution
Tooltip	Transaction data dependent condition may be specified here as enhanced expression. If this expression returns anything other than TRUE (ignore case) - this processor will be skipped.

---

4. Runtime algorithms

Topic	Behavior
Stream consumption	Original `InputStream` is closed during first `copyAndClose`
Cache sizing hint	`available()` used only as cache size hint — not a length guarantee
Overwrite policy	`FileOutputStream` truncates existing files — **no append mode**
Concurrency	No file locking; parallel routes writing the same path can corrupt output
Post-write stream	New stream from cache — callers must not reuse pre-processor stream reference
Attributes	Processor writes **no** transaction attributes

---

5. Worked examples

5.1 Archive after transform

Setting	Example value
Target Directory	`/data/out/%com.pilotfish.TradingPartner%`
Target File Name	`%OriginalFileName%.edi`

After execution: file exists on disk; downstream transport still receives the EDI bytes.

5.2 Conditional write

Set **Execute Processor** to an expression such as `%WriteToDisk% == 'true'` so only flagged transactions create files.

---

6. Known quirks and caveats

1. **Always overwrites** — existing files are replaced; no versioning built in.
2. **`available()` sizing** — very large streams may still work but cache allocation uses the hint only.
3. **No append / rotate** — single full-stream write per invocation.
4. **mkdirs on directory only** — intermediate paths inside `TargetFileName` rely on `File` parent creation behavior.
5. **Binary-safe** — uses stream copy; suitable for non-text payloads.
6. **Differs from File Transport** — this processor stays in-route and preserves the stream; transports typically hand off delivery.
7. **Error message is generic** — inspect cause for `IOException`, permission, or path errors.

---

7. Operational recommendations

Goal	Guidance
Unique names	Include transaction id, timestamp, or ISA control number in **Target File Name**
Directory layout	Use expression-driven subfolders under **Target Directory**
Idempotency	Pair with move/rename patterns if reprocessing must not clobber files
Downstream steps	Safe to place before transports that also need the payload
Permissions	Ensure the eiPlatform service account can create directories and write files
Testing	Verify expression-expanded paths in eiConsole test mode — empty names fail silently until runtime

---

8. Source index (this WAR)

Topic	Path under `war-pipeline/decompiled/eip-26R1.11/`
File Writing processor	`com/pilotfish/eip/modules/file/FileWriteProcessor`
AbstractProcessor	`com/pilotfish/eip/extend/AbstractProcessor.java`
Module JAR	`war-pipeline/jars/eip-26R1.11/xcs-modules-file-1.0`

---

9. Pipeline provenance

```
PilotFish WARs/eip.war.hs.26R1.11
  -> xcs-modules-file-1.0.1.jar
  -> CFR decompile
  -> Documents/Processors/26R1.11/PilotFish-File-Writing-Reference-26R1.11.(md|pdf)
```

Deep-dive documentation from WAR decompile – File Writing Processor – 26R1.11 – August 2026.